SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4304 *M.Madhu Rushwika(192371066)*
Marks:90

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

Code of Conduct:

I M.Madhu Rushwika(192371066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.Madhu Rushwika*

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understandi ng of Data	22.5	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	22.5	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	18	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	18	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	9	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1: **HTTP Response Analysis**

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

1. HTTP Response Analysis

Given HTTP response:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

1. Identify incorrect or invalid fields.

- `Content-Length: -150` is invalid because content length cannot be negative.
- The date format is valid, although the weekday should match the actual calendar date.
- The status code `200 OK` indicates that the request was successful.

2. Analyze the issue with Content-Length and explain its impact.

`Content-Length` specifies the size of the response body in bytes. A negative value is invalid and may cause the response to be rejected or treated as malformed by HTTP software.

3. Determine whether the status code matches the response correctness.

The status code `200 OK` indicates success, but it does not guarantee that every response header is valid. The invalid content length must be corrected.

4. Suggest corrections for the identified errors.

Replace the negative value with the actual response body size. For example, if the HTML body contains 150 bytes:

HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.

2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

2. DOM Structure Analysis

Given HTML:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

1. Identify missing or improperly closed elements.

- The `<p>` element is missing its closing `</p>` tag.
- The `<div>`, `<body>`, and `<html>` elements have closing tags.

2. Analyze how the browser will interpret this structure.

Browsers usually try to recover from common HTML errors. When the `</div>` tag is encountered, the browser can automatically close the open paragraph element.

3. Determine the impact on rendering and layout.

The page may still display correctly, but malformed HTML can cause unexpected spacing or layout differences and make the document harder to maintain.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld</p>
</div>
</body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method URL Example Data Visibility Security Level GET /login?user=admin&pass=1234 Visible in URL Low POST /login Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why. 3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET Method	POST Method
Sends data through the URL query string.	Sends data in the request body.
Data is visible in the URL.	Data is not normally visible in the URL.
Can be stored in browser history and logs.	Request body is not normally stored in browser history.
Not suitable for passwords.	More suitable for submitting login credentials.

2. Analyze which method is more suitable for login and why.

The POST method is more suitable for login because credentials are sent in the request body instead of the URL. However, POST alone does not provide encryption.

3. Identify security risks in using GET for authentication.

- Passwords may appear in browser history.

- URLs may be stored in server logs.
- URLs may be shared or copied accidentally.
- Credentials may be exposed through monitoring systems or other URL-handling mechanisms.

4. Suggest best practices for secure login implementation.

- Use POST to submit login credentials.
- Use HTTPS to encrypt communication.
- Store passwords using a secure password-hashing algorithm.
- Validate user credentials on the server.
- Use secure session cookies and apply rate limiting to prevent repeated login attempts.

Example:

```
<form action="/login" method="POST">
<label>Username:</label>
<input type="text" name="username" required>

<label>Password:</label>
<input type="password" name="password" required>

<button type="submit">Login</button>
</form>
```

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

4. HTML Code Inspection

Given HTML snippet:

```
<!DOCTYPE html>
```

```
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Identify missing closing tags.

- The `<h1>` tag is missing `</h1>`.
- The `<p>` tag is missing `</p>`.
- The `<img>` element is a void element, so it does not require a closing tag.

2. Analyze issues with the `<img>` tag.

- The image has no `alt` attribute for accessibility.
- The image may not display if `image.jpg` is missing or the path is incorrect.

3. Determine how browsers will handle these errors.

Browsers generally attempt to repair malformed HTML and display the content. However, missing tags can produce unexpected document structures or formatting.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph.</p>
<img src="image.jpg" alt="Sample image">
</body>
</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

5. Browser-Server Communication Flow

1. Interpret each step in the communication flow.

Step 1: Enter URL

The user enters a website address in the browser.

Step 2: DNS Lookup

DNS finds the IP address associated with the domain name.

Step 3: HTTP Request

The browser sends a request to the web server.

Step 4: Server Processing

The server processes the request and prepares the response.

Step 5: Response Returned

The server sends the HTTP response to the browser.

Step 6: Page Rendering

The browser processes HTML, CSS, images, and other resources to display the page.

2. Identify where delays can occur.

- DNS lookup may take time to resolve the domain.
- Network congestion can delay requests and responses.
- Slow server processing can increase response time.
- Large files and images can slow page loading.
- Complex HTML and CSS can increase rendering time.

3. Analyze the role of DNS in the process.

DNS stands for Domain Name System. It translates human-readable domain names, such as `example.com`, into IP addresses that computers use to communicate with servers.

4. Suggest optimizations to improve performance.

- Use DNS caching to reduce repeated lookups.
- Use a Content Delivery Network (CDN) to serve content closer to users.
- Optimize images and compress web resources.

- Use browser caching to reduce repeated downloads.
- Improve server performance and database queries.
- Use HTTP/2 or HTTP/3 where supported

Conclusion

HTTP response validation, correct DOM structure, secure login methods, valid HTML, and efficient browser-server communication are essential for developing reliable, secure, and fast web applications. These practices improve accessibility, security, and overall website performance.

Code of Conduct:

I M.Madhu Rushwika(192371066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M.Madhu Rushwika*

RUBRICS FOR CALCULATION:

Criteria	**Weigh tage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand